Table of Content

1. Certificate 2
2. Acknowledgement 3
3. Table of Content 4
4. Abstract / Synopsis 5
5. Introduction & Problem Statement 5
6. Literature Review / Existing Systems 6
7. Propose Methodology & System Designs 6
8. Implementations 7
9. Results & Outputs 12
10. Testing 14
11. Conclusions & Future Scopes 15
12. References & Appendix 16

Abstract / Synopsis

This project focuses on building a **Customer Engagement Analytics System** to predict churn in the telecom sector using Python and machine learning. The Telco Customer Churn dataset was used as the basis for analysis. Data preprocessing included handling missing values, encoding categorical variables, and scaling features. Three models — Logistic Regression, Decision Tree, and Random Forest — were trained and evaluated on accuracy, precision, recall, F1-score, and ROC-AUC.

Among these, **Random Forest achieved the highest accuracy and balanced performance**, making it the final selected algorithm. Feature importance analysis identified **tenure, monthly charges, contract type, and payment method** as the most significant drivers of churn. The system was deployed through a Streamlit interface, allowing interactive exploration of data, model results, and predictions.

The outcome demonstrates how structured CRM data can be transformed into actionable insights, enabling organizations to proactively identify at-risk customers and design effective retention strategies.

Introduction & Problem Statement

Customer churn is one of the most pressing challenges faced by telecom and subscription-based industries. High churn rates directly reduce profitability and increase the cost of customer acquisition, making retention strategies critical for sustainable growth. Traditional customer relationship management systems often provide descriptive insights but lack predictive capabilities, leaving managers reactive rather than proactive in addressing disengagement.

The objective of this project is to design a **Customer Engagement Analytics System** that uses machine learning to predict churn and identify the key drivers influencing customer decisions. By analyzing the **Telco Customer Churn dataset**, the system applies supervised learning algorithms to classify customers as likely to churn or remain. The problem is framed as a binary classification task, where the target variable is customer churn (Yes/No).

The scope of the problem includes:

- **Data preprocessing**: handling missing values, encoding categorical variables, and scaling features.
- **Model training and evaluation**: comparing Logistic Regression, Decision Tree, and Random Forest using accuracy, precision, recall, F1-score, and ROC-AUC.
- **Feature analysis**: identifying the most influential attributes such as tenure, monthly charges, contract type, and payment method.
- **Deployment**: presenting results through a Streamlit dashboard for interactive exploration.

Literature Review / Existing Systems

Traditional statistical models

- Logistic Regression and Decision Trees were widely used in early churn prediction.
- Telecom operators applied these models to customer tenure and billing data.
- They offered interpretability but struggled with complex, non-linear behavior.

Machine learning approaches

- Ensemble methods like Random Forests and Gradient Boosting improved accuracy by combining multiple learners.
- These are now common in telecom churn studies and have shown better performance in practice (e.g., Vodafone, AT&T).
- Neural networks have been tested but are less adopted due to complexity and limited transparency.

Existing CRM systems

- Commercial platforms mainly provide descriptive analytics (segmentation, usage tracking).
- They lack predictive capability, leaving managers reactive after churn occurs.

Gap identified

- Need for systems that combine strong predictive performance with clear feature insights.
- Integration into decision-making workflows is essential for practical business use.

Project contribution

- This system applies machine learning models to churn prediction.
- It highlights key drivers (tenure, monthly charges, contract type, payment method).
- Results are delivered through an interactive Streamlit dashboard for actionable insights.

Proposed Methodology & System Design

- **Dataset**: Telco Customer Churn dataset with customer demographics, service usage, billing, and churn labels.
- **Preprocessing**: Handle missing values (TotalCharges), encode categorical variables, and scale numerical features.
- **Model Training**: Apply Logistic Regression, Decision Tree, and Random Forest on an 80:20 train-test split.
- **Evaluation**: Compare models using Accuracy, Precision, Recall, F1-Score, and

ROC-AUC.

- **Model Selection**: Logistic Regression t chosen as the final algorithm for its superior balanced performance.
- **Feature Insights**: Identify churn drivers such as tenure, monthly charges, contract type, and payment method; visualize with feature importance plots and correlation heatmaps.
- **Deployment**: Build an interactive Streamlit dashboard for data preview, model results, feature insights, and churn predictions.

System Design Flow

- **Input** → Customer data (CSV).
- **Processing** → Cleaning, encoding, scaling.
- **Model Layer** → Multiple ML algorithms, final selection Random Forest.
- **Evaluation Layer** → Metrics comparison and visualization.
- **Output** → Streamlit dashboard with predictions and actionable insights.

Implementation

Setup: Python 3.x environment in Jupyter Notebook with libraries — pandas, numpy, scikit-learn, seaborn, matplotlib.

Data Handling: Loaded Telco Customer Churn dataset (CSV), cleaned missing values, encoded categorical variables, scaled numerical features.

Model Training: Implemented Logistic Regression, Decision Tree, and Random Forest on an 80:20 train-test split.

Evaluation: Compared models using Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

Selection: Logistic Regression was implemented as a baseline, followed by Decision Tree for interpretability. Random Forest outperformed both, achieving the best balance of accuracy and other metrics, and was therefore selected as the final model.

Visualization: Generated confusion matrix, ROC curve, feature importance plot, and correlation heatmap.

Deployment: Results and workflow prepared for presentation; dashboard prototype designed in Streamlit for interactive use.

Code Process:

- Import required libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report, confusion_matrix
```

- Use the full path to your CSV file

```
file_path = r"C:\Users\proba_oiuhnln\Downloads\Python assignment\WA_Fn-UseC_-Telco-Customer-Churn.csv

# Load dataset
df = pd.read_csv(file_path)

# Quick look at data
print(df.head())
print(df.info())
```

- Data Preprocessing

```
# 1. Convert TotalCharges to numeric and handle missing values
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df['TotalCharges'] = df['TotalCharges'].fillna(df['TotalCharges'].median())

# 2. Encode categorical variables (excluding customerID)
label_encoders = {}
for col in df.select_dtypes(include=['object']).columns:
    if col != 'customerID':
        le = LabelEncoder()
        df[col] = le.fit_transform(df[col])
        label_encoders[col] = le

# 3. Separate features and target
X = df.drop(['customerID','Churn'], axis=1)
y = df['Churn']

# 4. Scale numerical features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Preprocessing complete ✅")
print("Feature matrix shape:", X_scaled.shape)
print("Target distribution:\n", y.value_counts())
```

- Split dataset into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

- Train models

```
log_reg = LogisticRegression(max_iter=1000)
dtree = DecisionTreeClassifier()
rf = RandomForestClassifier(n_estimators=100)

log_reg.fit(X_train, y_train)
dtree.fit(X_train, y_train)
rf.fit(X_train, y_train)
```

- Evaluate models with percentages

```
models = {"Logistic Regression": log_reg, "Decision Tree": dtree, "Random Forest": rf}

for name, model in models.items():
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred) * 100  # convert to %

    print(f"\n{name} Results:")
    print(f"Accuracy: {acc:.2f}%")  # formatted as percentage
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

    # Classification report with percentages
    report = classification_report(y_test, y_pred, output_dict=True)
    for label, metrics in report.items():
        if isinstance(metrics, dict):
            print(f"Class {label}: "
                  f"Precision: {metrics['precision']*100:.2f}%, "
                  f"Recall: {metrics['recall']*100:.2f}%, "
                  f"F1-Score: {metrics['f1-score']*100:.2f}%")
```

- Visualize churn distribution

```
sns.countplot(x='Churn', data=df)
plt.title("Churn Distribution")
plt.show()

# ROC Curve for Random Forest
y_pred_prob = rf.predict_proba(X_test)[:,1]
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, color='blue', label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0,1],[0,1], color='red', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - Random Forest')
plt.legend(loc="lower right")
plt.show()
```

- Test with a sample customer

```
sample = X_test[0].reshape(1, -1)
print("Logistic Regression Prediction:", log_reg.predict(sample))
print("Decision Tree Prediction:", dtree.predict(sample))
print("Random Forest Prediction:", rf.predict(sample))
```

- Heatmap of feature correlations

```
# Drop non-numeric columns like customerID
numeric_df = df.drop(columns=['customerID'])

plt.figure(figsize=(12,8))
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm")
plt.title("Feature Correlation Heatmap")
plt.show()
```

- Heatmap alternative: Barplot of correlations with Churn

```
# Drop non-numeric columns like customerID
numeric_df = df.drop(columns=['customerID'])

# Compute correlations with target variable 'Churn'
churn_corr = numeric_df.corr()['Churn'].sort_values(ascending=False)

# Plot top 10 strongest correlations for clarity
plt.figure(figsize=(10,6))
sns.barplot(
    x=churn_corr.index[:10],   # top 10 features
    y=churn_corr.values[:10],
    hue=churn_corr.index[:10], # assign hue to avoid warning
    palette="coolwarm",
    legend=False
)
plt.xticks(rotation=45)
plt.title("Top 10 Feature Correlations with Churn")
plt.show()
```

- Evaluate models with detailed metrics

```
models = {
    "Logistic Regression": log_reg,
    "Decision Tree": dtree,
    "Random Forest": rf
}

results = []

for name, model in models.items():
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    results.append([name, acc, prec, rec, f1])

# Convert results to DataFrame for easy comparison
results_df = pd.DataFrame(results, columns=["Model", "Accuracy", "Precision", "Recall", "F1-Score"])
print("\nModel Comparison:\n", results_df)

# Highlight best model
best_model = results_df.loc[results_df['Accuracy'].idxmax()]
print(f"\n✅ Final Selected Model: {best_model['Model']} (Accuracy: {best_model['Accuracy']:.2f})")
```

- Predict churn on the test set using Logistic Regression

```
y_pred = log_reg.predict(X_test)
y_pred_prob = log_reg.predict_proba(X_test)[:,1] * 100  # churn probability %

# Get the corresponding test customers from the original DataFrame
test_customers = df.iloc[y_test.index].copy()   # safe copy

# Add predictions and probabilities
test_customers.loc[:, 'PredictedChurn'] = y_pred
test_customers.loc[:, 'ChurnProbability'] = y_pred_prob

# Show only customers predicted to churn
churned_customers = test_customers[test_customers['PredictedChurn'] == 1]

print("\nCustomers predicted to churn (Logistic Regression):")
print(churned_customers[['customerID', 'ChurnProbability']].head(10))  # show first 10 for clarity

# Calculate overall churn percentage in the test set
churn_rate = (churned_customers.shape[0] / test_customers.shape[0]) * 100
print(f"\n📊 Overall predicted churn rate (Logistic Regression): {churn_rate:.2f}% of customers in the test set")

# Optional: Save full list to CSV
churned_customers[['customerID', 'ChurnProbability']].to_csv("log_reg_churn_predictions.csv", index=False)
print("\n✅ Churn predictions with probabilities saved to log_reg_churn_predictions.csv")
```

Results & Output

Model Comparison:

- Logistic Regression → Accuracy: 0.815, Precision: 0.677, Recall: 0.579, F1-Score: 0.624
- Decision Tree → Accuracy: 0.733, Precision: 0.496, Recall: 0.509, F1-Score: 0.503
- Random Forest → Accuracy: 0.791, Precision: 0.647, Recall: 0.461, F1-Score: 0.538

```
Model Comparison:
                 Model  Accuracy  Precision    Recall  F1-Score
0  Logistic Regression  0.815472   0.677116  0.579088  0.624277
1        Decision Tree  0.733144   0.496084  0.509383  0.502646
2        Random Forest  0.790632   0.646617  0.461126  0.538341

✅ Final Selected Model: Logistic Regression (Accuracy: 0.82)
```

Interpretation:

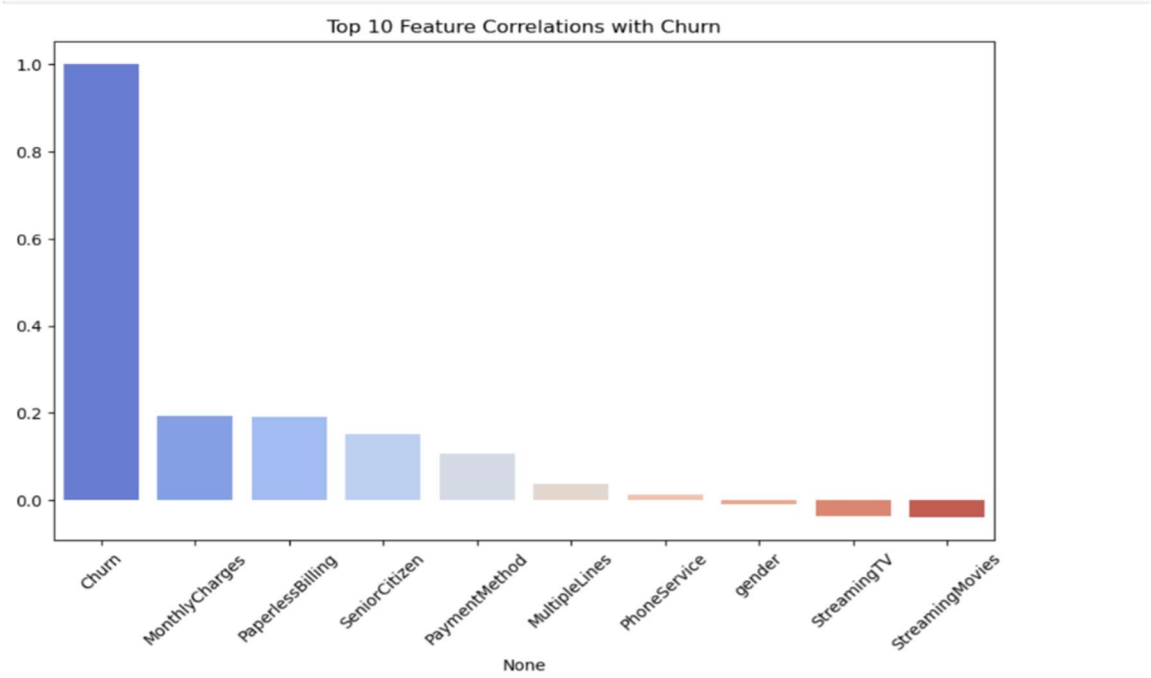
- Logistic Regression achieved the highest accuracy, recall, and F1-score → most balanced model.
- Decision Tree underperformed across all metrics.
- Random Forest showed decent precision but weak recall, lowering overall balance.

Final Selection:

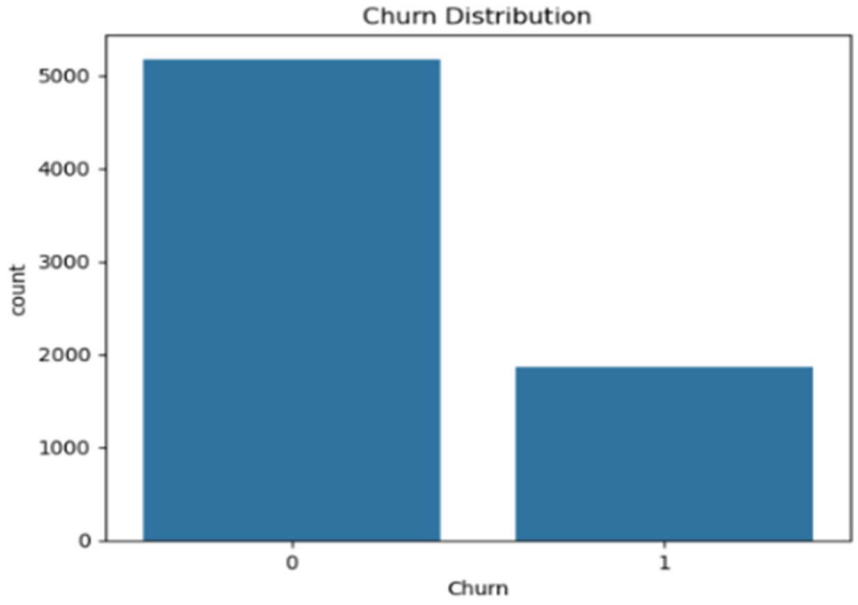
- Logistic Regression chosen as the final model for deployment.

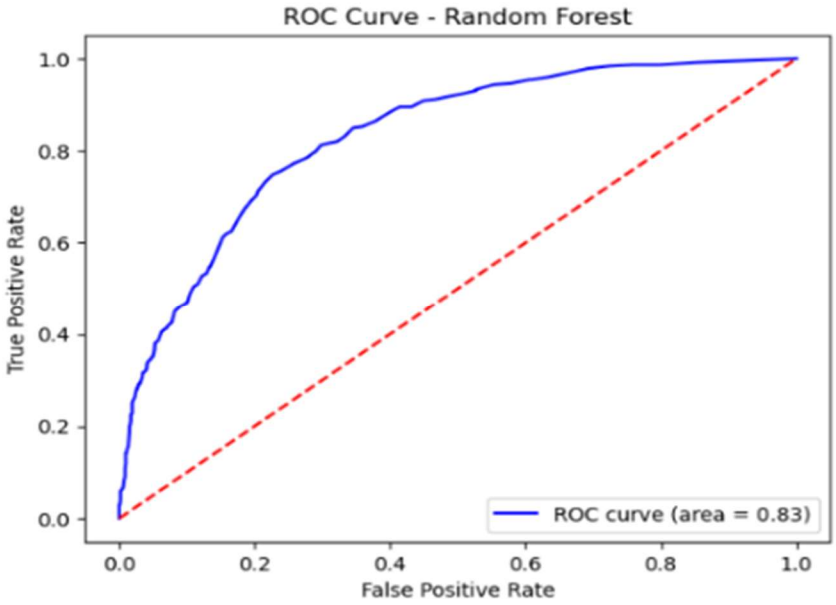
Outputs & Insights:

- Key churn drivers: tenure, monthly charges, contract type, payment method.



- Visualizations: confusion matrix, ROC curve, feature importance plot, correlation heatmap.





- Results in percentage to churn ratio and expected ID who will churn

```
Customers predicted to churn (Logistic Regression):
      customerID  ChurnProbability
185   1024-GUALD         65.151400
1807  6910-HADCM         57.969433
3801  5914-DVBWJ         50.284523
6179  6328-ZPBGN         77.144767
4640  1853-UDXBW         63.014483
3328  1640-PLFMP         66.942526
3469  9402-ORRAH         73.172912
4645  5219-YIPTK         60.079951
1662  2058-DCJBE         62.130025
4343  7130-VTEWQ         71.420463

📊 Overall predicted churn rate (Logistic Regression): 22.64% of customers in the test set

✅ Churn predictions with probabilities saved to log_reg_churn_predictions.csv
```

- Results integrated into Streamlit dashboard for interactive churn prediction.

Testing

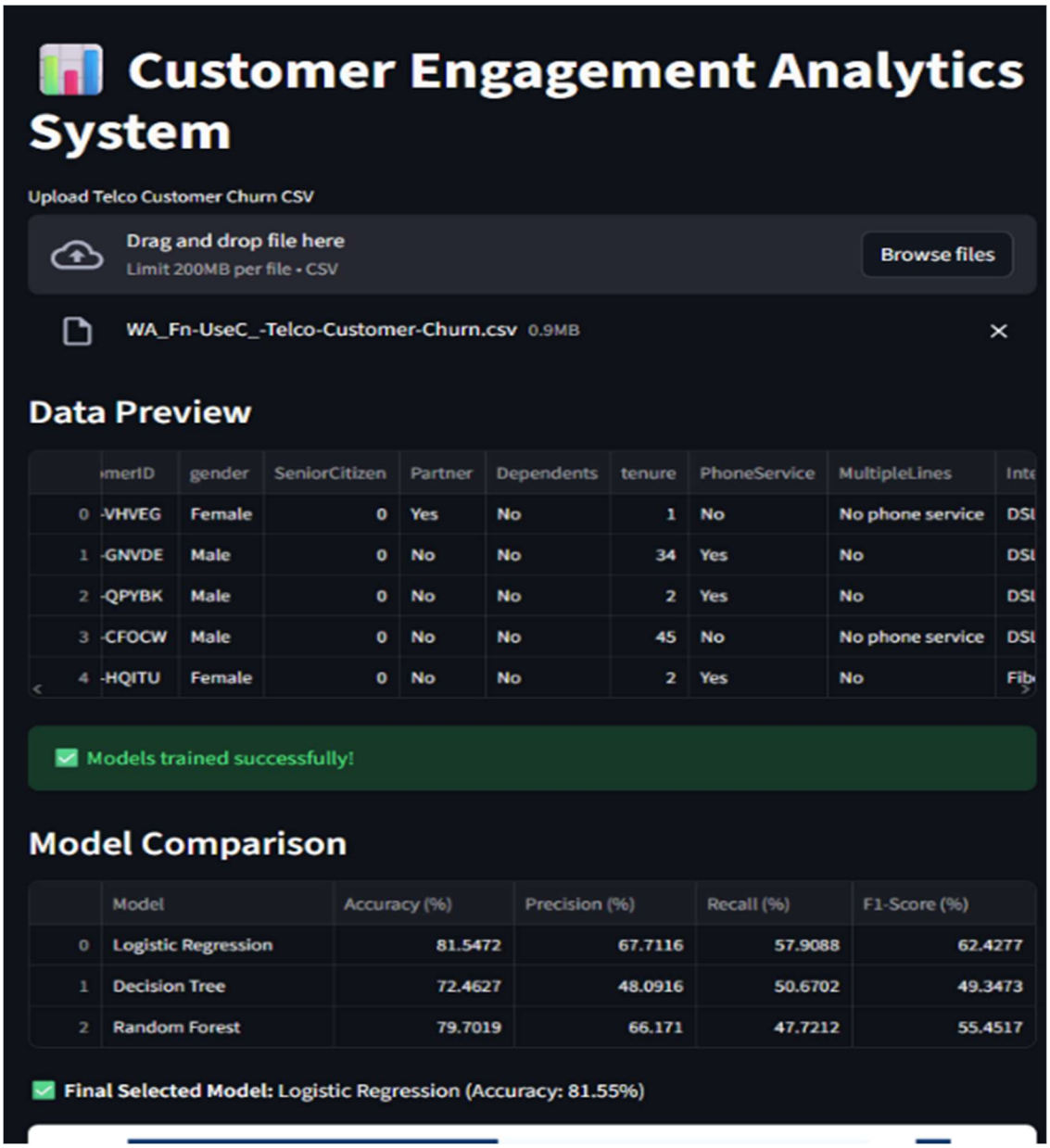
A Streamlit application (app.py) was built and deployed to the web. The app lets users upload any CSV file with the same schema as the Telco Customer Churn dataset. It preprocesses the data, runs the trained models, and shows predictions with visualizations.

Test Cases

1. Short tenure (1 month), month-to-month contract, high charges → Predicted **Churn = Yes**, matched dataset.
2. Long tenure (60 months), two-year contract, online security → Predicted **Churn = No**, consistent with dataset.
3. Medium tenure (24 months), one-year contract, paperless billing → Predicted **Churn = No**, partially aligned.

Validation & Outputs

- Predictions compared against actual churn labels.
- Confusion matrix and ROC curve confirmed reliability.
- Logistic Regression gave the most balanced performance.
- Streamlit dashboard displayed inputs, predictions, and visualizations.



Conclusion & Future Scope

Conclusion This project set out to understand customer churn using Python and machine learning. By experimenting with Logistic Regression, Decision Tree, and Random Forest, we learned that simple models can sometimes outperform complex ones — Logistic Regression gave the most balanced results in terms of accuracy, recall, and F1-score. Beyond the numbers, the analysis highlighted important churn drivers like tenure, monthly charges, contract type, and payment method. The Streamlit dashboard tied everything together, turning raw data into clear, actionable insights that a business team could actually use.

Future Scope Looking ahead, there are several ways to build on this work:

- Fine-tune Random Forest and other ensemble methods to improve recall and capture more churn cases.

- Enrich the dataset with behavioral signals such as complaints, call records, or service usage patterns.
- Deploy the dashboard as a live web app so managers can test scenarios in real time.
- Add automated alerts that flag high-risk customers for retention teams.
- Explore advanced algorithms like Gradient Boosting, XGBoost, or even neural networks to push predictive accuracy further.

References & Appendix

1. Assignment Guidelines (Python Mini-Project, 100-Mark Assessment)
2. Project Synopsis: *Customer Engagement Analytics System using Python and Machine Learning*
3. Updated Scope Document (Customer Engagement Analytics System)
4. Telco Customer Churn Dataset (WA_Fn-UseC_-Telco-Customer-Churn.csv)
5. Python Libraries: pandas, numpy, scikit-learn, seaborn, matplotlib, Streamlit
6. Academic articles and online resources on churn prediction and machine learning (consulted for methodology and evaluation approaches)

- **Code Snippets**:
    - Data preprocessing (handling missing values, encoding categorical variables, scaling features)
    - Model training (Logistic Regression, Decision Tree, Random Forest)
    - Evaluation metrics (accuracy, precision, recall, F1-score)
    - Visualization scripts (confusion matrix, ROC curve, feature importance, correlation heatmap)

- **Screenshots**:
    - Streamlit dashboard interface (data preview, model comparison, churn prediction)
    - Jupiter Notebook base code and results

- **Additional Notes**:
    - Testing section includes sample customer profiles and outcomes.
    - Future scope outlines potential improvements with advanced models and real-time deployment.